# Assignment: HBM Address Mapping and Bandwidth Analysis in Ramulator 2.0

**Weight:** 30 points · **Group size:** 4 · **Due:** 30th September 2026

## Goal

Peak bandwidth is set by the DRAM device; *achieved* bandwidth is set largely by the address mapping. You will measure how much bandwidth an HBM3 system loses when a power-of-two strided access pattern collides with Ramulator's default `RoBaRaCoCh` mapping, diagnose the collision at the level of individual address bits, then add a new C++ address mapper that XOR-hashes the channel index to recover the lost bandwidth.

**Simulator.** Ramulator 2.0, tag `v2.0a`, commit `be93be78055d922aa1d4d33e15bcc8f2b0c61a9d`. Pin this exact commit. The `main` branch has since been refactored (address mapping was split into a separate channel-mapper component) and this assignment's instructions will not match it. A C++20 compiler is required; `g++-12` and `clang++-15` are the versions the Ramulator authors test.

All file paths and line numbers below refer to that commit.

### Hardware context

- **Address mapping** turns a linear physical address into a DRAM coordinate vector (channel, pseudo-channel, bank group, bank, row, column). Which address bits feed which coordinate is the mapper's entire job.
- Ramulator's default mapper is `RoBaRaCoCh`. The name reads most-significant field first (Row at the top, Channel at the bottom), so **the channel index is taken from the lowest address bits**, just above the byte offset within a transaction. That placement works well for sequential streaming and very poorly for power-of-two strides.
- Ramulator's HBM3 model (`src/dram/impl/HBM3.cpp`) is simplified relative to the JEDEC part. Read the `org_presets` table and `m_internal_prefetch_size` and take the model's numbers as ground truth. Do not substitute datasheet values.

## Group organization (4 members)

## 1. Pre-release verification

Before releasing the assignment, build at the pinned commit and run Parts 0–1 end to end for at least one variant ($N = 16$, stride 256 KiB). Confirm that:

- `RoBaRaCoCh` sends all strided traffic to one channel, i.e. `num_read_reqs_<ch>` is non-zero for exactly one `ch`;

- the reference XOR mapper spreads that traffic across all $N$ channels.

The bit arithmetic in Parts 2 and 3 hangs on two facts, both of which must be re-verified if the assignment is moved off tag `v2.0a`:

1. `m_tx_offset = 4`, i.e. 16 B transactions (prefetch 2× 64-bit channel);
2. the `RoBaRaCoCh` field order is `channel, column, pseudochannel, bankgroup, bank, row`.

## 2. Group formation (58 students)

With 58 students there are 14 groups of 4 with 2 students left over. Place the two remaining students so that there are either two groups of 3 or two groups of 5. The work-package table degrades gracefully in both directions:

- **Group of 3:** one person takes Part 0 **and** the report assembly (WP-A merged with the collective report duty).
- **Group of 5:** the fifth member becomes a second owner on WP-D, which is the critical path.

Assign every group a number so that the per-group variant table (group number mod 4) resolves for all groups.

## 3. Variant table (reference)

| Group no. mod 4 | Channels $N$ | Stride | XOR shift $k$ |
|---|---|---|---|
| 0 | 16 | 256 KiB | 0 |
| 1 | 16 | 512 KiB | 1 |
| 2 | 8 | 256 KiB | 1 |
| 3 | 32 | 512 KiB | 0 |

Four work packages, one primary owner each. The Part 3.3 synthesis and the report are collective. Your contribution statement (last page of the report) must name the work package each member owned.

| WP | Primary owner delivers | Part | Can start once... |
|---|---|---|---|
| **WP-A** | Build at the pinned commit; source inventory; **assembles the report** | Part 0 | immediately |
| **WP-B** | Trace generation, config files, baseline stream/strided runs, bandwidth and peak math | Part 1 | WP-A build works |
| **WP-C** | Bit-level diagnosis: derive the channel bits, predict channel usage, confirm against counters | Part 2 | WP-B has baseline data |
| **WP-D** | Implement the XOR mapper, rebuild, re-measure both traces | Part 3.1–3.2 | WP-A build works |
| **all** | Part 3.3 joint analysis; report assembly; contribution statement | Part 3.3 | WP-C and WP-D |

Each stride is a power of two that (a) is far larger than the channel-interleave granule, so it fully collides under `RoBaRaCoCh`, and (b) lands exactly on the row field, so the Part 3 XOR has clean bits to fold in. Results are expected to differ across groups in magnitude; the mechanism is the same.

# 4. Part 0 — Setup (3 points)

```
git clone https://github.com/CMU-SAFARI/ramulator2
cd ramulator2
git checkout be93be78055d922aa1d4d33e15bcc8f2b0c61a9d
mkdir build && cd build && cmake .. && make -j && cp ./ramulator2 ../ramulator2 &&
    cd ..
./ramulator2 -f ./example_config.yaml
```

The build produces a `ramulator2` executable and a `libramulator.so`. The example run uses the `SimpleO3` core model on `example_inst.trace`; it should print a block of YAML statistics and exit 0.

**Report (1 pt).** The commit hash you built, your compiler and version, and the tail of the example run's statistics output.

**Report (2 pts).** List the address mappers registered in `src/addr_mapper/impl/linear_mappers.cpp` (there are three) with a one-line description of each, taken from the registration macro and the `apply()` body. Then answer, citing file and line:

1. Does the HBM3 model expose **pseudo-channel** as an explicit level in the address vector, or is the stack modelled purely as channels? (Look at `m_levels` in `src/dram/impl/HBM3.cpp`.)
2. Which component turns the linear address into the coordinate vector, and which entry of that vector does the memory system use to pick a channel controller? (Follow `send()` in `src/memory_system/impl/generic_DRAM_system.cpp`.)

Your answers fix what Part 2 and Part 3 are allowed to target.

# 5. Part 1 — Baseline bandwidth (7 points)

## 5.1 Traces (1 pt)

The trace frontend you will use is `LoadStoreTrace` (`src/frontend/impl/memory_trace/loadstore_trace.cpp`). Confirm its line format by reading `init_trace()`: each line is one token `LD` or `ST`, a space, then one address (decimal, or `0x`-prefixed hex). It feeds the address to the address mapper, which is what this assignment is about. Do **not** use `ReadWriteTrace`, which expects a pre-decoded comma-separated coordinate vector and bypasses the mapper entirely.

Generate two traces with the provided `tracegen.py`:

```
python3 tracegen.py --pattern seq    --n 30000 --granularity 16 --out traces/stream.
    trace
python3 tracegen.py --pattern stride --n 30000 --stride 262144  --out traces/strided
    .trace
# use your assigned stride
```

- `-granularity 16` for the stream is the HBM3 transaction size (derived below); a dense 16 B stride exercises every channel in turn.
- The stride trace does not wrap. With 30 000 requests it stays inside device capacity for every variant.

### 5.2 Config (2 pts)

Write `configs/hbm3.yaml`. Every implementation name below is the string in a `RAMULATOR_REGISTER_IMPLEMENTATION` macro in the source; verify each one:

```
Frontend:
  impl: LoadStoreTrace           # src/frontend/impl/memory_trace/loadstore_trace.
    cpp
  path: ./traces/stream.trace
  clock_ratio: 1

MemorySystem:
  impl: GenericDRAM              # src/memory_system/impl/generic_DRAM_system.cpp
  clock_ratio: 1
  DRAM:
    impl: HBM3
    org:
      preset: HBM3_8Gb
      channel: 16                # your assigned N (override of the preset's 1)
    timing:
      preset: HBM3_2Gbps
  Controller:
    impl: Generic                # the one generic controller; no per-standard
    controller
    Scheduler:      { impl: FRFCFS }
    RefreshManager: { impl: AllBank }
    RowPolicy:      { impl: OpenRowPolicy }
  AddrMapper:
    impl: RoBaRaCoCh
```

Run the strided trace by copying this file and changing `path:` to `./traces/strided.trace`, or override on the command line:

```
./ramulator2 -f ./configs/hbm3.yaml \
  -p "Frontend.path=./traces/strided.trace" \
  -p "MemorySystem.DRAM.org.channel=16"
```

Note in your report that `AllBank` refresh issues no refreshes for HBM3 in this model: it iterates `get_level_size("rank")`, which is $-1$ for HBM3, so your runs are refresh-free. State this in your report.

### 5.3 Transaction size and clock (1 pt)

Ramulator prints neither bytes nor nanoseconds. Derive both from the source:

- **Transaction bytes** $B_{tx}$ = `m_internal_prefetch_size` $\times$ `m_channel_width` $/\,8$. Read both fields for HBM3 (`src/dram/impl/HBM3.cpp`; `m_channel_width` defaults in `set_organization()`). Show the arithmetic. This is also $2^{\texttt{m_tx_offset}}$, where `m_tx_offset` is computed in `LinearMapperBase::setup()` (`src/addr_mapper/impl/linear_mappers.cpp`).

- $t_{CK}$ in ns = `m_timing_vals("tCK_ps")` / 1000, and `tCK_ps` is recomputed in `set_timing_vals()` as $10^6$/(rate/2) from the timing preset's `rate`. Show it.

### 5.4 Measurements (3 pts)

Run both traces. From the statistics YAML, take `memory_system_cycles`, `total_num_read_requests`, and the per-channel counters `num_read_reqs_<ch>`, `row_hits_<ch>`, `row_misses_<ch>`, `row_conflicts_<ch>`, `avg_read_latency_<ch>`.

$$\text{BW (GB/s)} = \frac{\texttt{total_num_read_requests} \times B_{tx}}{\texttt{memory_system_cycles} \times t_{CK,\text{ns}}}$$

$$\text{avg read latency (ns)} = \frac{\sum_{ch} \texttt{read_latency}_{ch}}{\sum_{ch} \texttt{num_read_reqs}_{ch}} \times t_{CK,\text{ns}}$$

$$\text{row-buffer hit rate} = \frac{\sum_{ch} \texttt{row_hits}_{ch}}{\sum_{ch} (\texttt{row_hits}_{ch} + \texttt{row_misses}_{ch} + \texttt{row_conflicts}_{ch})}$$

**Sum the per-channel counters across all channels; do not read channel 0 and multiply.** In the strided run only one channel is active, so its counter *is* the total; in the stream run all $N$ are active.

| Trace | BW (GB/s) | Avg read latency (ns) | Row-buffer hit rate | # channels with traffic |
|---|---|---|---|---|
| stream | | | | |
| strided | | | | |

Also compute the **peak** bandwidth of your configured device. Peak is set by data-bus occupancy: a column access holds a data bus for `nBL` cycles (the constraint below, found in `HBM3.cpp`), and the independent data buses number channels × pseudo-channels. So

$$\text{peak} = \frac{(N \times \text{pseudo_channels}) \times B_{tx}}{\texttt{nBL} \times t_{CK,\text{ns}}},$$

with `nBL` from the timing preset. State every bandwidth number's scope: per channel, or summed over all buses.

The timing constraint referred to above is:

```
{.level = "pseudochannel",
 .preceding = {"RD","RDA"}, .following = {"RD","RDA"},
 .latency = V("nBL")}
```

## 6. Part 2 — Diagnosis (8 points)

### 6.1 Bit layout and channel collision (4 pts)

Read `RoBaRaCoCh::apply()` in `src/addr_mapper/impl/linear_mappers.cpp` together with `LinearMapperBase::setup()`. Using your $N$ and the `HBM3_8Gb` preset, write the physical-address bit layout produced by `RoBaRaCoCh`: the bit range assigned to each of tx-offset, channel, column, pseudo-channel, bank group, bank, and row. Show how you got each width (`calc_log2` of the level count; the column width is reduced by $\log_2$(`m_internal_prefetch_size`)).

Then:

- State the exact bit range of the **channel** field.
- Show what those bits do as the address advances by your assigned stride $S$: since $S$ is a power of two and every bit below $\log_2(S)$ is constant, and the channel bits sit entirely below $\log_2(S)$, the channel index never changes.
- Predict the fraction of the $N$ channels the strided trace touches, and confirm it against `num_read_reqs_<ch>` for every channel. Report prediction and measurement.
- The pseudo-channel bit sits just above the column field and is also below $\log_2(S)$ for your stride, so it is pinned too. Say which pseudo-channel every strided request lands in, and what that costs on top of the channel collapse.

### 6.2 Why the sequential trace is unaffected (4 pts)

The sequential trace does not show the problem. Explain why in terms of the same bit layout: a 16 B stride increments the channel field by one every request, so requests round-robin across all $N$ channels. Then state the general rule: why channel bits at the **low** end of the address are ideal for unit-stride streaming and very poor for any power-of-two stride at least as large as the channel-interleave granule ($N \times B_{tx}$ bytes).

## 7. Part 3 — C++ modification: XOR-based channel hashing (12 points)

`RoBaRaCoCh` is a pure field-slicer. The same file also contains `MOP4CLXOR`, which XOR-hashes low-order index bits with column bits. Study it as the pattern for how a non-trivial mapper is written and registered. Your mapper folds a slice of the **row** bits into the **channel** index instead.

### 7.1 Implement (6 pts)

Add a new mapper. The registration pattern is in the repository README, under "Adding an Implementation to an Existing Interface".

1. Add a class next to the others **inside** `src/addr_mapper/impl/linear_mappers.cpp`. That file is already in `src/addr_mapper/CMakeLists.txt`, so no CMake change is needed; this is the simple path. If you instead create a new `.cpp`, add it to `target_sources` in that `CMakeLists.txt`.
2. Inherit from **both** `LinearMapperBase` (which provides the `IAddrMapper` interface and `setup()`) and `Implementation`.
3. Put `RAMULATOR_REGISTER_IMPLEMENTATION(IAddrMapper, YourClass, "YourName", "...")` *inside* the class body.
4. Select it from YAML with `AddrMapper: { impl: YourName }`.

The logic: in your `apply()`, run the exact body of `RoBaRaCoCh::apply()` first, so that `req.addr_vec` holds the sliced channel and row indices, then append one line that hashes the channel index with a slice of the row field:

```
req.addr_vec[0] ^= (req.addr_vec[m_row_bits_idx] >> k)
                   & (m_addr_bits[0] > 0 ? ((1 << m_addr_bits[0]) - 1) : 0);
```

with $k$ your assigned shift (`m_addr_bits[0]` is the channel bit count, so the mask is $N-1$). Justify $k$ from your Part 2 layout: your stride is $2^{(\text{row_start_bit}+k)}$, so `row_value >> k` has your

access index $i$ in its low $\log_2(N)$ bits, and the XOR turns "always channel 0" into a round-robin over all $N$ channels.

**Bijection.** A mapping is only legal if every physical address still maps to exactly one DRAM location. XOR-ing the channel field with a value that depends only on the row field is a bijection because XOR by a fixed operand is its own inverse: given the output channel and the (unchanged) row you recover the input channel exactly. Note that addition modulo $N$ with $N$ a power of two would *also* be a bijection here, since it is a rotation of residues; the reasons hardware uses XOR are that it is self-inverse and carry-free, not bijectivity. What breaks bijectivity is a non-invertible combiner: bitwise AND/OR, or multiplication by a constant that shares a factor with $N$. Explain this in two or three sentences.

Also show, from your Part 2 arithmetic, that your row-bit slice is **not constant** under your stride. Otherwise the XOR does nothing.

### 7.2 Re-measure (2 pts)

Re-run **both** traces on HBM3 with your mapper. Extend the Part 1.4 table with two rows (`stream + XOR`, `strided + XOR`). Report what you measure, including any regression on the stream.

### 7.3 Analysis (4 pts)

1. Explain your bitwise logic: which row bits you folded into the channel index, and why those bits (and not others) carry the access-pattern entropy.
2. Explain architecturally *why* strided bandwidth changed, citing the per-channel `num_read_reqs_<ch>` counts and the row-buffer counters before and after, rather than intuition.
3. Report the effect on the **sequential** trace. If it is unchanged, explain why the XOR is harmless there (the row bits barely move over a dense stream, so the XOR operand is $\approx 0$ and the mapping $\approx$ identity). Then state, in one sentence, the residual bottleneck your channel hash does *not* fix. Hint: look at which bank each strided request lands in within its channel.

## Deliverables

Submit `group<N>_hbm.tar.gz`, one per group:

```
report.pdf        # <= 4 pages; first page names each member and the WP they owned
src/              # your mapper class (the modified linear_mappers.cpp)
                  #   and any CMakeLists change
configs/          # every YAML you ran
results/          # raw simulator stdout, one file per run
README.md         # commit hash, compiler + version, exact trace + run commands,
                  #   repro steps
```

No trace files; submit the commands that regenerate them.

Every bandwidth number needs a stated scope. Every "why" claim in Part 2 and Part 3 needs a counter behind it. A **contribution statement** on the last page gives one paragraph per member naming their work package. All members are responsible for the whole report and may be asked about any section.

## Rubric

| Part | Pts |
|---|---|
| 0 — Build, mapper inventory, pseudo-channel / channel-selection questions | 3 |
| 1 — Transaction-size and clock derivation, baseline stream vs. strided, peak calculation | 7 |
| 2 — Bit-layout derivation, channel-collision prediction confirmed against counters | 8 |
| 3 — XOR mapper: implementation and bijection argument, re-measurement, analysis | 12 |
| **Total** | **30** |

**Deductions:** bandwidth without a stated scope (−2); results not reproducible from your README (−3); missing contribution statement (−3); Part 2 or Part 3 claims asserted without the supporting counters (−3); editing a DRAM standard file instead of the mapper (−3).

## Pitfalls

1. `memory_system_cycles` is in memory-clock cycles; multiply by $t_{CK,\text{ns}}$ for time. The `clock_ratio` of the Frontend and MemorySystem must be equal (use `1` and `1`), or the trace frontend and the DRAM run at different rates and every number rescales.
2. Address mapping lives in `src/addr_mapper/`. `HBM3.cpp` holds organization and timing only; do not edit it.
3. There is exactly one general controller, registered as `Generic` (`src/dram_controller/impl/generic_dram_controller.cpp`). HBM3 uses it. There is no separate "HBM controller" to match.
4. If your new mapper is not selected, check that the string in the registration macro matches the YAML exactly, and (if you made a new file) that it is listed in `src/addr_mapper/CMakeLists.txt`.
5. `avg_read_latency_<ch>` is per channel and is `nan` for a channel with no reads. Aggregate over channels as shown in Part 1.4.
6. Bandwidth above your computed peak means the peak calculation or the cycle count is wrong.